PFP VIII: Conservative Grammar Growth and Source-Bound Evidence in the MSS Pipeline

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Research methods paper · Author-review revision

Code baseline: 52cb5457f6c485ba548cfd292f6012a941ed22cf

Abstract

Adding a new symbol, checking a finite example, and proving a semantic theorem are different achievements. This paper studies how the Proposal Fidelity Protocol (PFP, historically the Symbol Game) records those distinctions in the MSS grammar and its evidence pipeline. MSS-N denotes stage N in the packet-language catalog. The implemented context maps preserve the single old packet at the MSS-0 to MSS-1 transition and return a typed non-descent receipt for new structure. That limited round trip establishes injectivity on its declared domain, but does not establish a general tower or a conservative extension of a theory. We connect this example to computed stage grades, finite Lean declarations, and a versioned resolver that separates historical Git bytes from installed resources. A worked flow fixture shows why matching objective values can coexist with an invalid certificate. We then apply the same standard to a challenge corpus and to recorded model runs. A census at a stated commit sorts 50 challenges into delivered solver packages, model measurements, and specifications, and corrects earlier counts. In recorded runs of two small models on one graph, the correct repair appeared only when the prompt contained the repair values, and the host's acceptance rule could also be met by discarding the claims. The contribution is a source-pinned methods case study: explicit preservation obligations, adversarial controls, and evidence-custody checks that a reader can reproduce without accepting a stage name or a success label as proof. The deposit includes the actual initial parser and maps, two retained working reports, and all eight K4 traces with full revision and hash bindings. A 16-question study companion turns these examples into predictions and changed-input checks.

1. What must survive an extension?

Suppose a language initially has one legal packet, written as `<>`. A later language adds named packets such as `<0>` and `<1>`. Two requirements are easy to confuse. First, the old packet should remain identifiable: storing it in the new language and reading it back should recover the same object. Second, its old meaning and all claims made about it should remain justified. The first requirement alone does not establish the second.

This distinction matters when a research system grows by adding stages. A stage number may organize a catalog, but it cannot certify that the stage's advertised mathematics has been implemented. Likewise, a successful Lean compilation establishes propositions expressed in Lean; it does not automatically establish the broader meaning suggested by a file's title.

We examine four layers in the pinned source: actual packet syntax and context maps, stage-specific executable checks, formal declarations and their build evidence, and retrieval of the exact bytes cited by a result. The central question is operational: what may a reader conclude at each layer, and what observation would invalidate that conclusion?

This is a methods and audit paper. It does not introduce an infinite mathematical tower, infer physical laws from a grammar, or report a controlled study of model reasoning.

2. The packet language and the implemented round trip

The archived grammar module uses the following recursive syntax:

```
packet ::= diff | atom | app
diff   ::= <>
atom   ::= <word>
app    ::= op<packet*>
word   ::= one or more ASCII characters from a-z or 0-9
```

Packets are self-delimiting. An application can contain adjacent child packets without commas. The empty distinction `<>` has its own Diff constructor; it is not an Atom with an empty name. Parsing produces a structured value; a separate level-admission predicate decides which packets are active in a particular context.

For the context-map module, the only MSS-0 packet is Diff. The module updown_maps.py implements precisely u_0 and d_1. The upward map first checks that its input is active at level zero, then returns that same packet. The downward map returns an old packet unchanged and returns a NonDescentReceipt containing the original packet otherwise.

Let $S_0 = \{\text{Diff}\}$, and let S_1 be the intended active MSS-1 packet domain. Its interface is better represented by

$$u_0 : S_0 \longrightarrow S_1, \qquad d_1 : S_1 \longrightarrow S_0 \sqcup \text{Refusal}, \qquad d_1(u_0(x)) = x.$$

The disjoint alternative matters: a refusal is not an ordinary old packet. Calling the down-map a total function from S_1 to S_0 would erase the distinction that the implementation deliberately preserves.

Input operation	Result	What it establishes
`d_1(u_0(parse("<>")))`	Diff	Old packet is recovered exactly
`u_0(parse("<0>"))`	ValueError	Non-old input violates the upward precondition
`d_1(parse("<1>"))`	NonDescentReceipt retaining `<1>`	New structure is not silently converted into Diff

These are the concrete cases in the context-map tests. The d_1 function itself accepts a typed Packet and tests old-level membership; it does not independently certify that every supplied packet belongs to S_1. A caller needing that guarantee must run the level validator.

Left-inverse fact. If $d(u(x)) = x$ on an old domain, then u is injective there: applying d to $u(x) = u(y)$ gives $x = y$. In the implemented transition the old domain has one element, and the identity follows directly from the two function branches. This is a useful API invariant, with a deliberately small mathematical scope.

The first part of the package-local test file test_008_mssn_tower_retraction.py is a separate teaching toy. It changes a record's level field and uses a string-prefix rule to return None for a

new term. Its example does not implement the domain maps above, and None is not their typed refusal. We retain that toy as historical material; the paper's implementation claims refer to the actual modules.

3. Why injectivity is weaker than semantic conservativity

A conservative extension of a theory must preserve its old consequences: if the extended theory proves a sentence in the old language, that sentence must already follow from the original theory. A round trip on packet objects does not mention derivability, truth, or an interpretation. Verified compilation shows the stronger kind of statement: the correctness theorem of the CompCert backend relates the observable behaviors of a source program and its compiled code (Leroy, 2009). A syntactic round trip makes no such comparison.

For a simple counterexample, keep every old term unchanged while adding a new inference rule that declares an unproved old-language proposition P to be an axiom. The identity inclusion of terms still has a left inverse. The extended proof system nevertheless has a new old-language consequence. Consequently, injective syntax cannot justify semantic conservativity.

An extension that intends to preserve meaning needs additional data: an interpretation for both languages, a compatibility statement on embedded old terms, and a theorem that the allowed new rules do not create unsupported old conclusions. A stronger operational claim might also require preservation of accepted transitions, failure states, and observable outputs. None of these follows by an increase in an integer stage field alone.

The audited code baseline does not supply a family of u_n and d_n maps with these properties for all stages. Its source explicitly restricts the concrete context-map implementation to MSS-0 and MSS-1. Larger catalogs and proposed obligations remain useful design material, but should be cited as such.

One namespace fact does generalize. The registry allocates eight consecutive proposal opcodes starting at

$$b(N) = 128 + 8(N - 22), \qquad N \geq 22.$$

For $N < M$, $b(N) + 7 < b(M)$; hence these integer intervals are disjoint. The source opcodes are below 128, so the proposal blocks also avoid the declared source namespace. Python's unbounded integers support this formula; a fixed-width wire encoding would require an additional upper bound. The loaded catalog covers stages zero through 100, not every natural number. Neither the arithmetic formula nor the catalog constructs those stages' semantics.

4. Evidence grades come from checks

The stage-status module uses the grades NONE, SYNTAX, and COMPUTED. It does not grant a grade because a level lies below a chosen threshold.

A SYNTAX check round-trips the stage's own packet through serialization, admits it at its declared stage, and refuses it one stage early. For stage zero, the negative probe is an interior glyph that should not yet be admitted. These checks are evidence of the selected syntax boundary. They do not prove the entire parser correct for every input.

A COMPUTED binding identifies a checker and exact fixtures. The checker must accept its positive example and refuse its stage-specific negative example and empty input; floating-point evidence is excluded. Registered target checkers additionally bind checker bytes and the stage-row digest, and exercise near misses, mutations of each deciding clause, and applicable probes with their expected refusal codes.

For example, the finite-section binding covers $\{a, b\}$ and $\{b, c\}$. Assignments $(a = 0, b = 1)$ and $(b = 1, c = 0)$ glue to $(a = 0, b = 1, c = 0)$. Replacing the second value at b by zero creates a conflict and must fail. This validates a finite dictionary operation; it does not establish a general sheaf theorem.

Registering a checker says which executable is responsible for a stage. Running it supplies the current grade. Tests that pass on a small fixture family still leave untested inputs, specification mistakes, and common errors shared by a checker and its fixtures. The grade is therefore a bounded evidence label, not a universal soundness theorem.

This status interface never assigns LEAN or GATED. Lean evidence belongs to a separate pipeline. A stale comment that no symbol-game axiom report has been captured must not be read as a repository-wide inventory: the Level Engine has its own report artifacts and custody requirements.

5. A formal fixture that exposes an invalid certificate

At the code baseline, the Level Engine directory contains 52 payload modules plus its axiom-report module. Their audited source is in lean4/research_programs/level_engine. Their statements must be read individually.

Consider FlowCertificate.lean. It defines a three-edge directed cycle, unit penalties, edge loads $c = (1, 0, 0)$, and zero vertex levels. Its primal absolute-residual cost is one. With flows $y = (1, 1, 1)$, the dual dot product is also one, capacity bounds hold, and the flow balances at every vertex.

Now change only the flow to $y = (1, 0, 0)$. The dot product remains $c^{\mathsf{T}} y = 1$, so the reported gap is still zero. Capacity bounds also remain true. But the divergence, defined as inflow minus outflow, is $(-1, 1, 0)$. This vector cannot serve as a feasible dual circulation.

The declarations explicitly prove that the own-negative fixture has zeroGap equal to true and balanced equal to false. Thus a theorem about one arithmetic clause can be true even when the proposed certificate fails as a whole. Paper 001 supplies the general weak-duality argument explaining why every certificate clause is needed.

The file uses exact-rational definitions and kernel-decided finite propositions. It does not by itself prove a general optimization theorem over all graphs. Similarly, the following source readings place limits on several suggestive filenames. Each item gives what the file states or constructs and the further conclusion it does not establish.

- Level16IndObject.lean: natural-number capacities 2^{n+1} increase and exceed every finite bound. Not established: a category, a colimit construction, or a theorem about embeddings of that colimit.
- Level21DiscreteRegge.lean: concrete rotation commutator has zero scalar exponent sum and nonidentity matrix product. Not established: general discrete Regge dynamics.
- Level22FibredDependentContexts.lean: reindexing copies a record's fields; identity and composition hold for that operation. Not established: a Grothendieck fibration with its universal lifting property.

- Level23SheafDescent.lean: restrictions of a global function are compatible; conflicting restrictions prevent amalgamation. Not established: a general existence-and-uniqueness gluing theorem.

The revised Level21 file expressly withdraws itself as evidence for its earlier MSS-021 claim and formalizes a finite counterexample instead. That correction must survive later summaries. A build result cannot restore the withdrawn interpretation.

Copies of these five files are vendored in this package's `lean4/` folder (Lake file), together with the ExactRat library that FlowCertificate.lean imports. Each copy is byte-identical to its source at the code baseline apart from a two-line provenance comment. The retained author receipts record their compilation with Lean 4.34.0, and the axiom report lists the axioms of all 39 theorems the five files declare. Thirty-eight depend on no axiom beyond propext and Quot.sound. The theorem stage_capacity_monotonic in the Level16 file also depends on Classical.choice. The two-axiom policy of Section 6 belongs to the Level Engine audit and governs only its payload modules; the Level16 file is not one of them.

The broader local report lists 576 environment declarations from ten modules, including ExactRat and generated or private declarations. This is not a count of manuscript theorems. The retained October 2 author receipt records a successful local build and this broader census. Review checked its 13 protected source hashes, the full census source and report, and the census tool against the deposited bytes; all match. A fresh Lean replay in the present review host is blocked: the official, digest-verified Lean 4.34.0 binary exits before startup with "failed to locate application". The retained source bindings remain inspectable, but this review does not claim an independent kernel pass.

6. Fresh formal evidence needs custody

A proof assistant checks a proposition under its definitions and assumptions. Lean's documentation explains the role of extensionality, quotients, and choice in that logical environment. Official Lean reference.

The Level Engine build driver pins Lean 4.34.0 and records source hashes, return codes, and whether the sources remained stable. Its axiom-audit driver requires a successful build receipt matching the current source bytes and toolchain before running a fresh report.

The report must cover the complete generator-declared theorem census without missing, extra, or duplicate names. The local policy permits only propext and Quot.sound. It rejects dependencies such as sorryAx, Classical.choice, or a custom unproved axiom, and compares the fresh report with the canonical report. This is the package's chosen policy; a theorem using classical choice is not thereby a mathematical falsehood.

At the pinned baseline, the driver checks source hashes before and after Lean returns. Review found a later publication gap: a source edit during census inspection can be recorded under an earlier successful audit because the publisher recomputes its expected hashes. A bounded mocked reproduction returned success and published the changed source; the downstream snapshot reader correctly rejected its mismatched build/audit identities.

The local correction in the verification patch carries the audited source map through the receipt and snapshot writer, refuses a post-audit edit, and validates copied bytes against that map. The regression tests exercise this control logic with fake Lean responses. They do not establish a fresh

Lean build, filesystem locking, or protection against an edit that is made and reverted between observations.

The custody tests use isolated fake tool responses to check these failure paths. They include a changed source after a success receipt, an absent receipt, an incomplete declaration census, and disallowed axioms even when report replacement is requested. They test the control logic. They are not fresh Lean kernel elaborations of the 52 payloads.

Neither this review nor the retained local build rebuilt the 52 external Level Engine payloads. The local 39-theorem selection and the 576-declaration environment census are different scopes; neither certifies those payloads. A new full Level Engine result requires its own source-bound execution receipt.

The deposit's optional named Lean gate also compares the complete expected declaration names and axiom sets with the replay. An empty successful process, a missing, extra or duplicate declaration, or a changed dependency is refused. Mocked process tests validate this gate logic and do not compile a theorem.

7. Resolve the cited bytes, not a convenient replacement

An old evidence path may no longer exist in the working directory. Substituting a current file with the same name can quietly change the evidence while leaving the citation looking intact.

The versioned resolver has two deliberately separate read modes:

Mode	Required source	Missing-data behavior
v1	Explicit repository, full 40-character revision, canonical relative path	HISTORICAL_BYTES_UNAVAILABLE
v2	Installed package resource under the active v2 namespace	ACTIVE_RESOURCE_UNAVAILABLE

The historical mode invokes Git's object reader for the exact revision-and-path blob; it does not fall back to the working tree. The active mode uses package resources and refuses a historical revision argument. Both reject absolute paths, parent traversal, backslashes, empty segments, and other noncanonical forms. An optional SHA-256 check rejects a mismatch. A caller that omits this digest has not obtained that additional content check.

The resolver tests create a temporary Git repository, commit bytes containing line endings and a zero byte, remove the working file, and read the original bytes through v1. They also check wrong digests, unavailable revisions, missing repository context, noncanonical paths, and misuse of the v2 revision argument. Git's documented object-content operation supplies the underlying retrieval primitive. Git cat-file documentation.

This mechanism preserves byte identity under its stated assumptions. It does not establish who authored the bytes, whether a checker is correct, or whether a mathematical claim follows from them. The caller must still bind each cited artifact to the proposition and source revision being reviewed.

8. A benchmark label is not an observation

The historical 150-test runner contains a concrete evidence failure: some absent-receipt cases receive VERIFIED, caught provider exceptions can also receive VERIFIED, and some route or metric values are fixed records rather than measurements from the named experiment. The aggregate counts PASSED and VERIFIED alike.

Therefore an aggregate 150/150 record from that procedure cannot establish 150 independently executed successful scientific checks. Nor can it support the former manuscript's claims of "perfect governed resolution", "zero ungoverned completion", or a measured retraction frequency. Those claims are withdrawn. The recorded spend and duration are not adopted as validated benchmark results.

A sound replacement requires recording which operation actually ran, preserving raw outputs, distinguishing missing and failed execution from success, and applying an independent outcome oracle to each experimental claim. The controlled-evaluation design belongs to Paper 003. Sections 9 and 10 apply the same standard to the challenge corpus and to recorded model runs.

9. The challenge corpus at a stated commit

At the census revision, omega_engine_projects/mssn_sn_symgam/challenges/ holds challenge specifications written against one protocol, SG-CP v1. This paper audits C001 to C050, in five subfolders: mechanism (C001 to C010), cohomology (C011 to C020), quantum information (C021 to C030), physics (C031 to C040), and formal mathematics (C041 to C050). The folder also holds C051 to C100 and G01 to G25, which are outside this paper.

The protocol asks each challenge for a participant packet and a judge packet. The judge packet holds hidden cases, relabel and gauge controls, poisoned but schema-valid cases, an exact checker, a second checker written separately, and a mutation suite: corrupted outputs that the checker must reject. The protocol admits only open-weight models with at most 70B total parameters. It sets a per-arm spending ceiling by tier, from $0.10 at tier T1 to $5.00 at tier T5, enforced before each call. It asks for a committed pre-registration and exact McNemar tests on paired cases. It requires exact arithmetic and allows floating-point checks only as labeled secondary evidence.

9.1 Census

We classify every challenge from the tree of commit a623568eb0f853b968bc419bf06dfc6af256ab02 (2026-09-28). The challenge folder is read at that historical revision; it is not identified with today's main branch. The three categories are disjoint:

- **A. Delivered solver with two checkers**: a reference solver, two checker modules in the judge packet, and executable checks.
- **B. Model measurement deliverable**: the specification calls for a measurement of a language model rather than a reference solver. Membership does not assert that a measurement was completed.
- **C. Specification only**: a specification exists and no reference solver is delivered.

Subfolder	A	B	C
Mechanism	none	C001, C005	C002 to C004, C006 to C010
Cohomology	C011 to C020	none	none

Subfolder	A	B	C
Quantum information	C021 to C030	none	none
Physics	C031 to C040	none	none
Formal mathematics	C041 to C050	none	none

Category A therefore holds 40 of the 50 challenges. C051, outside the audited range, is a forty-first delivered package. The census file lists every Category A folder with its two checker modules, its number of test files, hidden cases, and mutation files, and its retained scoring receipt. File counts are not pytest collection counts. C047 has one test-named file, judge_packet/test_cases_runner.py, which prints scores for its cases using both checkers, without assertions or a failing exit status; it does not check the mutation files. C048 to C051 have no pytest test file in the inventory. Their scoring programs must be read for their actual scope; their presence does not establish that every hidden case and mutation was executed. The C014 receipt records that its scoring was invalidated and a rescore is pending. No scoring receipt is retained for C015 to C020 or C045 to C051. Membership in Category A records what a folder contains; it does not certify that a package is correct.

The previous revision listed C001 and C005 in both Categories A and B, and it listed C012 to C020, C022 to C030, C032 to C040, and C043 to C050 as specifications without solvers. That list was wrong even at the code baseline: packages for C012 to C020 and C032 to C040 were already present at commit `52cb5457`. Packages for C022 to C030 and C041 to C051 were added after it, on 2026-09-27 and 2026-09-28.

9.2 Retained author checks

The September 28 author ledger records five test-file runs from three Category A packages, one file per command, with Python 3.12.10 and pytest 8.3.5. All five are recorded as exit status 0. The present review inspected the historical census and retained source; it did not rerun these remote challenge packages.

Package and test file	Result
C011 judge_packet/mutation_suite/test_checker_mutations.py	41 passed: an inventory check, and 40 mutation files each rejected by both checkers
C011 tests/test_dual_checkers.py	5 passed
C021 judge_packet/mutation_suite/test_checker_mutations.py	41 passed: an inventory check and 40 mutation files
C031 judge_packet/mutation_suite/test_checker_mutations.py	47 passed: an inventory check, 45 mutation files, and a check on refusal mutations
C031 tests/test_reference_solver.py	9 passed

These runs and the retained files correct earlier figures. C011 has 46 judge cases (32 base cases, 8 invariance controls, and 6 poisoned cases) and 40 mutation files, not 32. Its checkers verify an exact dual certificate for minimum weighted L_1 coboundary repair: the primal cost $\sum_e w_e\,|c_e - (\delta_0\phi)_e|$ equals $\sum_e c_e z_e$ for a circulation z with zero divergence at every vertex and $|z_e| \le w_e$. C031 has 45 mutation files, not 36, and the earlier count of 71 tests is not traceable to the retained files; it is withdrawn. Its reference-solver tests check that no chiral solution exists for $n = 3$ or $n = 4$ within the charge bounds 3, 5, 10, and 12, and that for $n = 5$ with $|q_i| \le 9$ the enumerator returns exactly

one canonical solution, $(-9, -5, -1, 7, 8)$. Up to sign and order this is $(1, 5, -7, -8, 9)$, for which $1 + 5 - 7 - 8 + 9 = 0$ and $1 + 125 - 343 - 512 + 729 = 0$. The earlier figure of 38 end-to-end tests for C021 was the number of test functions in the shared folder `challenges/e2e_tests/`, which this revision did not run; it is withdrawn. C021's second checker includes a floating-point eigensolver that the package labels as secondary evidence.

We did not run the C041 checkers, since each case needs a Lean call. By source inspection, they refuse a fixed list of tokens that includes `sorry`, `axiom`, `unsafe`, `set_option`, and `native_decide`, and they accept a proof only when its axioms lie in {propext, Quot.sound}. Proofs by `native_decide` depend on the additional axiom `Lean.ofReduceBool`. The package records Lean 4.34.1 as its toolchain; the earlier text named 4.34.0.

9.3 Category B: model measurements

C001 contains a harness that runs Qwen2.5-0.5B-Instruct with real weights, and a 200-case McNemar module that describes itself as a synthetic fixture with injected baseline failures. No measurement result from the harness is retained in the repository: it writes its report only to an untracked run folder, and it lists the full 200-case sweep as its next test. The earlier statement that C001 evaluated distractor-key masking across 200 tasks is withdrawn.

C005 compares prompt and output formats on 50 graph-consistency cases (25 consistent, 25 with a planted cycle defect) with one model, nousresearch/hermes-3-llama-3.1-405b, a fine-tune of Llama 3.1 405B, at temperature 0. The host's exact integer checker scored every answer: the verdict had to match the case label, and a CONSISTENT verdict needed a potential that satisfies every edge. The results come from the retained archived C005 working report, which describes two September 27 runs and retains its September 29 correction notice. Its old aggregate wording is not adopted here.

Run	Arm	Prompt and output format	Correct
Three-arm run	1	Relations in words; verdict first	25/50
Three-arm run	2	Relations in words; numeric scratchpad before the verdict	36/50
Three-arm run	3	Precomputed spanning-tree potentials and chords; chord residuals only	19/50
Hybrid run	2	As in the three-arm run	38/50
Hybrid run	4	Factorized input, a canary field, and a scratchpad; the canary field must also be right	4/50

In the paired comparisons below, b counts cases correct only under the first arm and c counts cases correct only under the second. We computed the exact two-sided binomial McNemar p-value from b and c. For $n = b + c > 0$, we use

$$p = \min\left(1,\ 2\,2^{-n}\sum_{k=0}^{\min(b,c)}\binom{n}{k}\right),$$

and set $p = 1$ when $n = 0$. These are descriptive paired comparisons, without a multiple-comparison adjustment.

Comparison and run	b	c	Exact p	Approximate p
Arm 1 vs Arm 2, three-arm run	6	17	$\frac{145499}{4194304}$	0.0347
Arm 1 vs Arm 3, three-arm run	6	0	$\frac{1}{32}$	0.0313
Arm 2 vs Arm 3, three-arm run	20	3	$\frac{1}{2048}$	0.00049
Arm 2 vs Arm 4, hybrid run	34	0	2^{-33}	1.2×10^{-10}

Within each run the counts and the paired tables agree: $25 - 6 + 17 = 36$, $25 - 6 = 19$, $36 - 20 + 3 = 19$, and $38 - 34 = 4$. The previous revision reported Arm 2 as 37/50 (74.0%), the midpoint of the two runs, and set it beside tables taken from both runs; no run produced 37/50. The per-case rows were written to an untracked run folder and are not in the repository, so the counts above are transcribed from the report and could not be recomputed from per-case data here.

The C005 runs also depart from the protocol. The C005 specification and SG-CP v1 both limit models to open weights with at most 70B total parameters and set a ceiling of $0.10 per arm. The runs used a 405B model and a ceiling of $2.00 per benchmark run. They are reported here as a deviation from the protocol, not as a conforming C005 result. On this one model and case set, the two factorized-input arms scored below the verdict-first arm; that observation does not support a general statement about prompt or schema complexity.

10. Recorded retraction traces

10.1 Eight K4 traces

The eight retained traces are archived in docs/source_archive/k4_traces/. The accompanying working benchmark report (last changed 2026-09-25) records a deterministic replay through the host. Its custody table gives SHA-256 digests for eight traces, one for each pair of model and condition. We recomputed all eight digests and they match. There is one trace per cell, so every rate below is 0/1 or 1/1. The earlier figure of 28 trials, and the per-cell counts of 4 and 3 derived from it, are not traceable to retained files and are withdrawn.

All eight runs use one fixture: the complete graph K_4 on vertices a, b, c, d, with cycle rank 3, OpenRouter as the provider, temperature 0, and 12 turns. The model asserts the six edge claims and then proposes repair moves. After each move the host recomputes its defect value Φ_1, which is zero exactly when the current claims admit a potential. A repair move replaces the whole claim set with the claims it lists, an omitted edge counting as zero, and the host admits it unless Φ_1 increases. A session is accepted when the model verifies at $\Phi_1 = 0$ or when the session ends at $\Phi_1 = 0$. The

correct repair keeps the three tree edges and sets the chords to c→a = −3, b→d = −3, and c→d = −5. The four conditions differ in what the host's prompt supplies once all six claims are asserted:

Condition	What the prompt supplies beyond the graph
Baseline	The sum around each of the three basis cycles, and the one-word guidance "qualitative"
Partially masked	The spanning-tree potentials and a formula for each chord value; the model must do three subtractions
Scratchpad	As partially masked, plus a worked example whose arithmetic yields exactly −3, −3, and −5
Hinted	The target value of each chord and a complete recommended repair move

The scratchpad and hinted prompts therefore contain the repair values. They are assisted conditions: a pass shows that a model copied or reproduced supplied values, not that it found the repair unaided.

Model	Condition	Turns	Loop count	Final Φ_1	Outcome
Qwen2.5-7B-Instruct	Baseline	12	6	8	Not accepted
Qwen2.5-7B-Instruct	Partially masked	12	5	6	Not accepted
Qwen2.5-7B-Instruct	Scratchpad	9	1	0	Accepted
Qwen2.5-7B-Instruct	Hinted	9	1	0	Accepted
Llama-3.1-8B-Instruct	Baseline	12	5	5	Not accepted
Llama-3.1-8B-Instruct	Partially masked	12	4	7	Not accepted
Llama-3.1-8B-Instruct	Scratchpad	12	4	0	Accepted
Llama-3.1-8B-Instruct	Hinted	12	4	0	Accepted

The loop count follows the counting rule of the benchmark script, which the deposit does not include; its retained last version is research/mssn_sn_tower/demos/demo_1/benchmark_open_weights_nullspace.py at commit 7b0d2903a13ceada014f572e4a3f9073ad37f95a. A turn counts when the host classifies a move as stationary or refuses it for raising the defect, or when the model asserts an edge after turn 6. The rule therefore also counts Qwen's seventh assertion, which was needed because it asserted the first edge twice. The count is a label from that rule, not a count of identical repeated proposals. Each trace also records the host's replies to every turn except the last, and those replies are consistent with the retained replay table. The final row has no stored post-move host result. The present review verified all eight trace hashes and inspected their proposals; the terminal defect values above remain the historical report's replay, not a fresh replay or a final field read from each raw trace.

Three details of the traces qualify the table. First, both accepted Qwen runs proposed the correct repair at turn 8 and verified at turn 9. Second, both Llama runs in the assisted conditions proposed the correct repair at turn 7, moved away from it, and proposed it again later (turn 11 with the scratchpad, turn 12 with the hint); both were accepted by the end-of-session rule. The scratchpad run's last move set the three chords to zero and omitted the tree edges; the host admits such a move, which leaves $\Phi_1 = 0$ on a claim set of zeros. Third, Llama's turn-9 move in the baseline

condition was of the same kind. The host labeled it an exact repair, although it replaced the claims with zeros rather than repairing them; later assertions restored the defect.

These records support one narrow statement. On this fixture, neither model proposed the correct chord values when the prompt left the three subtractions to it, and both models proposed them when the prompt contained them. Acceptance was decided by the host's exact check of Φ_1, not by the model. That check scores the claims that a move proposes, not their agreement with the claims asserted earlier, so an acceptance label is weaker than a repair of the asserted claims.

The previous revision said that hinted prompts reduced loops to at most 1.0 and that hinted runs converged within an average of 8.8 to 11.0 turns. With one trace per cell and Llama's hinted loop count of 4, those figures are withdrawn. We also do not describe the hinted condition as supplying only a boundary vector: it supplied the full repair. The same report contains further traces for Qwen2.5-72B-Instruct and Llama-3.3-70B-Instruct on the complete graphs K_4, K_5, and K_6; this paper does not analyze them.

10.2 Hamiltonian traces

The previous revision reported that the engine reached exact-cancellation terminals on three two- and four-site Hamiltonians that it called stoquastic, and that it correctly emitted a sign-obstruction terminal for a three-site triangular Hubbard plaquette. The retained traces are in research/mssn_sn_tower/demos/demo_2/traces/ and in Paper 001's `results/` folder. The repository's own audit of those runs and the Demo 2 notes found that the historical runners set the defect to zero after one asserted matrix entry and accepted a named symmetry without checking the proposed map. Their exact-cancellation terminals are therefore false acceptances, not certificates, and the claim is withdrawn.

The triangular trace, demo2_r5_frustrated_triangular_1790208154.jsonl, records a different pattern. The host refused the model's sign-free claim with a message stating that the triangle is an odd cycle, told the model to declare an obstruction, and recorded the obstruction terminal that the model then declared. The fact behind that refusal is that a triangle has no bipartition: no assignment of signs η to the three sites satisfies $\eta_u \eta_v = -1$ on every edge. That is a statement about one family of site gauges. The prompt declared hopping and on-site terms but no particle number, basis ordering, or path representation, so the trace supports no classification of the Hamiltonian's sign problem. The source of the rule that the host applied is not retained with the trace. The baseline's check for a related question, sign_switching.py, takes a real symmetric matrix in a declared basis and decides whether some diagonal ± 1 change of basis makes every off-diagonal entry nonpositive; when none exists, it returns a cycle whose positive entries have odd parity. The earlier sentence about diamond graphs and "infinite 2-cycles" had no retained receipt at the stated scope and is withdrawn.

10.3 The package runner

The package's runner is a smaller, single-shot tool, separate from the recorded sessions above. It asks whether a potential exists on six fixed graphs under four conditions. Each host verdict has a certificate: a potential satisfying every edge, or a closed walk with nonzero signed claim sum. A potential telescopes to zero on every closed walk, so the latter certificate rules out all potentials. The verifier also accepts walks loaded back from JSON and refuses malformed or altered walks.

Scoring accepts an exact CONSISTENT or CONTRADICTION verdict; malformed explicit labels, conflicting labels, and substring matches yield UNKNOWN. Before querying the proposer, the

runner refuses a fixture whose declared label disagrees with the host. Each version-2 execution receipt binds the complete input graph, exact prompt, raw reply, provider label, measurements, host certificate, and declared runner-source digest. Its public SHA-256 detects changes relative to a trusted expected digest; it does not authenticate a provider or prove a certificate, since an editor can recompute it.

The hinted prompt gives the host's edge residuals, which decide the answer. The offline stub reads the prompt; it matches 3/6, 6/6, 3/6 and 6/6 under baseline, hinted, scratchpad and exact-oracle conditions. These are plumbing controls, not model measurements. The local scratchpad asks for generic subtraction reasoning; it is different from the historical K4 scratchpad's answer-bearing example. A repeated-proposal helper counts the longest consecutive run of repeated comparisons, not a total pooled across separate runs. It is not the historical K4 loop rule.

Offline mode opens no network connection or credential file. Outputs go to an explicit directory or a new temporary directory outside the package. Live mode requires an environment key and reports no run here. Its nominal $0.10 budget uses a fixed $0.001 next-call estimate and reported cost; it is not a guaranteed bound on provider charges.

11. Reproduction and practical use

Use Python 3.12 from this package directory, one test file per command. Install the versions in the review receipt if reproducibility of these checks matters:

```
py -3.12 -m pip install pytest==9.1.1 ruff==0.14.13 mypy==1.19.1
py -3.12 -m pytest code/tests/test_008_mssn_tower_retraction.py -q
py -3.12 -m pytest code/tests/test_008_deposit_gates.py -q
py -3.12 -m pytest code/tests/test_008_source_bound_examples.py -q
py -3.12 -m ruff check code
py -3.12 -m mypy
py -3.12 code/retraction_benchmark_runner.py --mock --condition hinted --output-dir <new-
    directory>
py -3.12 tools/replay_lean_axiom_census.py --check .
py -3.12 tools/replay_lean_axiom_census.py --replay .
```

On Unix systems, use the same commands with the Python 3.12 interpreter in place of `py -3.12`. Lean replay requires a working Lean 4.34.0 toolchain; a failed startup is a blocked replay, not a mathematical result. The source example tests import the archived initial grammar and maps in an isolated namespace. They also recompute the flow fixture and the four exact McNemar values, and compare certificates for all 125 integer triangle assignments with edge values from -2 to 2 against an independent bounded potential search. This is a finite test domain, not a proof of a general graph algorithm.

The quiz and worked study guide connect each calculation to its source, scope limit and changed-input control.

The old repository-root commands refer to the code baseline, whose directory layout has since changed. To replay those historical checks, use a separate checkout at that full commit; use the census commit for the challenge packages. Historical Level Engine generation, build and audit likewise need that checkout, its dependencies and vendor inputs. They are separate from the standalone deposit checks and are not executed by this review.

The approach separates declaring a syntax extension, checking a finite instance, and asserting general semantic preservation.

12. Scope of the claims

This paper supports the following statements.

1. The MSS-0 to MSS-1 context maps form a left inverse on a one-element domain and return a typed non-descent receipt for new packets. A left inverse gives injectivity; it does not give a conservative extension.
2. At commit `a623568e`, 40 of the challenges C001 to C050 are delivered solver packages with two checker modules, 2 call for model measurements, and 8 are specifications only. The retained author runs for C011, C021 and C031 record rejection of their 40, 40 and 45 mutation files respectively.
3. The C005 report's counts, read per run, agree with its paired tables, and the exact p-values follow from its discordant counts. The runs departed from the protocol's model-size and budget limits.
4. In eight recorded K4 traces, one per model and condition, the two 7B and 8B models proposed the correct repair exactly in the conditions whose prompts contained the repair values, and the host's acceptance rule could also be met by a move that replaced the claims with zeros.
5. Retained source-bound author receipts record a Lean 4.34.0 build of the local formal package. Its 39 selected theorems and the broader 576-declaration census have different scopes. This review verifies those source bindings and reports its fresh local Lean replay as blocked.

This paper does not support the following statements.

1. That an infinite tower, or a conservative extension for every stage, exists.
2. That any Category A package is correct beyond its retained receipt and the historical author runs, or that the Category C specifications are implemented.
3. Any measured C001 result, any general claim about prompt formats from C005, or any reading of the C005 runs as a protocol-conforming result.
4. Unassisted repair by the models, or any rate estimated from single traces.
5. Any classification of the sign problem for the Hamiltonian traces, or any polynomial-time solution of general sign problems.

The narrowest defensible claim is this. In the recorded runs, the host's exact check of Φ_1 decided which proposals were accepted; the prompts changed only what the models proposed. On one K4 fixture, the two small models proposed the correct repair only when the prompt contained the repair values.

The next test named by the previous revision, reference solvers for C012 and C022, is no longer open: both packages are delivered and hold retained scoring receipts. The open next test is an unassisted comparison: several seeds on the K4 fixture with prompts that give the potentials but not the chord values, against a matched control, under a pre-registered analysis, together with publication of per-case C005 rows so that its tables can be recomputed.

13. Limitations

The census records folder contents at one historical commit. Section 9.2 retains the author's five challenge-test runs; this review did not replay them. The C005 counts rest on a report whose per-case rows are not retained. The K4 traces are single runs on one fixture at temperature 0, and their loop count comes from one script's rule. The Hamiltonian traces support no physical conclusion. The Level Engine payloads were not rebuilt. Retained local Lean source identities match, but the fresh review-host replay is blocked. The 16-question curriculum is a self-study design, not a

measured educational intervention.

This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against the manifests. The publication review and source-bound review receipt record this review's offline tests, lint, typing, source comparisons, PDF build and blocked fresh Lean replay. They distinguish executed checks from retained historical outcomes. The older receipts keep their original dates and source hashes; unchanged Lean bindings remain relevant, while their old runner and teaching-material hashes do not describe the revised package.

References

- Avigad, Jeremy, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich, with contributions from the Lean Community. "Axioms and Computation." Chapter 12 of *Theorem Proving in Lean 4*. Official online book; accessed 2026-09-28. Chapter; book and authors.
- Git project. "git-cat-file — Provide contents or details of repository objects." *Git Reference Manual*, online edition (release 2.56.0 when accessed on 2026-09-28). Manual page.
- Leroy, Xavier (2009). "A Formally Verified Compiler Back-end." *Journal of Automated Reasoning*, 43(4), pp. 363–446. doi:10.1007/s10817-009-9155-4; author preprint, arXiv:0902.2137.